

Multimodal Visual Localization Application

Multimodal Visual Localization Application

1. Concept Introduction
 - 1.1 What is "Multimodal Visual Localization"?
 - 1.2 Brief Description of the Principle
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largetmodel/utils/tools_manager.py`)
 2. Model Interface Layer (`largetmodel/utils/large_model_interface.py`)
 - Code Analysis
3. Practical Operation
 - 3.1 Configure Online LLM
 - 3.2 Start and Test the Function

1. Concept Introduction

1.1 What is "Multimodal Visual Localization"?

Multimodal visual localization is a technology that combines multiple sensor inputs (such as cameras, depth sensors, IMU, etc.) and processing algorithms to achieve precise identification and tracking of a device or user's position and posture in the environment. This technology does not rely on a single type of sensory data but integrates information from different perceptual modes, thereby improving the accuracy and robustness of localization.

1.2 Brief Description of the Principle

1. **Cross-modal Representation Learning:** To enable the LLM to process visual information, a mechanism needs to be developed to convert visual signals into a form that the model can understand. This may involve using convolutional neural networks (CNNs) or other architectures suitable for image processing to extract features and map them to the same embedding space as text.
2. **Joint Training:** By designing appropriate loss functions, text and visual data can be trained simultaneously within the same framework, allowing the model to learn to establish connections between these two modes. For example, in a question-answering system, an answer can be given based on the provided text question as well as the relevant image content.
3. **Visually-guided Language Generation/Understanding:** Once an effective cross-modal representation is established, visual information can be used to enhance the functionality of the language model. For example, when given a photo, the model can not only describe what is happening in the picture but also answer specific questions about the scene, and even execute instructions based on visual cues (such as navigating to a certain location).

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utlils/tools_manager.py`)

The `visual_positioning` function in this file defines the execution flow of the tool, especially how it constructs a Prompt containing the name of the target object and format requirements.

```
# From largemodel/utlils/tools_manager.py
class ToolsManager:
    # ...
    def visual_positioning(self, args):
        """
        Locate object coordinates in image and save results to MD file.

        :param args: Arguments containing image path and object name.
        :return: Dictionary with file path and coordinate data.
        """
        self.node.get_logger().info(f"Executing visual_positioning() tool with
args: {args}")
        try:
            image_path = args.get("image_path")
            object_name = args.get("object_name")
            # ... (Path fallback mechanism and parameter checking)

            # Construct a prompt asking the large model to identify the
coordinates of the specified object.
            if self.node.language == 'zh':
                prompt = f"请仔细分析这张图片，用一个个框定位图像每一个{object_name}的位
置..."
            else:
                prompt = f"Please carefully analyze this image and find the
position of all {object_name}..."

            # ... (Build independent message context)

            result = self.node.model_client.infer_with_image(image_path, prompt,
message=message_to_use)

            # ... (Process and parse the returned coordinate text)

            return {
                "file_path": md_file_path,
                "coordinates_content": coordinates_content,
                "explanation_content": explanation_content
            }
        # ... (Error handling)
```

2. Model Interface Layer

(`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image-related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
        # ... (Prepare message)
        try:
            # Decide which specific implementation to call based on the value of
            self.llm_platform
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Logic for calling Tongyi model
                pass
            # ... (Logic for other platforms)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code Analysis

The core of the visual localization function lies in **guiding the large model to output structured data through precise instructions**. It also follows the layered design of the tool layer and the model interface layer.

1. Tool Layer (`tools_manager.py`):

- The `visual_positioning` function is the business core of this function. It receives two key parameters: `image_path` (image path) and `object_name` (name of the object to be located).
- The core operation of this function is to **construct a highly customized Prompt**. It not only simply requests the model to describe the picture, but also embeds the `object_name` into a well-designed template, explicitly requiring the model to "locate the position of each {object_name} in the image," and implicitly or explicitly requiring the result to be returned in a specific format (such as a coordinate array).
- After constructing the Prompt, it calls the `infer_with_image` method of the model interface layer, passing the image and this customized instruction together.
- After receiving the returned text from the model interface layer, it also needs to perform **post-processing**: using regular expressions and other methods to parse the precise coordinate data from the model's natural language reply.
- Finally, it returns the parsed structured coordinate data to the upper-level application.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function still plays the role of a "dispatch center." It receives the image and Prompt from `visual_positioning` and dispatches the task to the correct backend model implementation according to the current configuration (`self.llm_platform`).
- For the visual localization task, the responsibility of the model interface layer is basically the same as that of the visual understanding task: correctly package the image data and text instructions, send them to the selected model platform, and then return the returned text result to the tool layer without modification. All platform-specific implementation details are encapsulated in this layer.

In summary, the general flow of visual localization is: `ToolsManager` receives the target object name and constructs a precise Prompt that requests the return of coordinates -> `ToolsManager` calls the model interface -> `model_interface` packages the image and the Prompt together and sends them to the corresponding model platform according to the configuration -> the model returns text containing coordinate information -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` parses the text, extracts the structured coordinate data, and returns it. This process demonstrates how to use Prompt Engineering technology to make a general visual large model complete more specific and structured tasks.

3. Practical Operation

3.1 Configure Online LLM

1. **Update the key in the configuration file and open the model interface configuration file** `large_model_interface.yaml`:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. **Fill in your API Key:**

Find the corresponding section and paste the API Key you just copied. Here is an example of Tongyi Qianwen configuration.

```
# large_model_interface.yaml

## Tongyi Qianwen
qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key
qianwen_model: "qwen-vl-max-latest" # You can choose the model as needed,
such as qwen-turbo, qwen-plus
```

3. **Open the main configuration file** `yahboom.yaml`:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

4. **Select the online platform to use:**

Modify the `llm_platform` parameter to the name of the platform you want to use.

```
# yahboom.yaml

model_service:
  ros__parameters:
    # ...
    llm_platform: 'tongyi' # Optional platforms: 'ollama', 'tongyi',
    'spark', 'qianfan', 'openrouter'
```

After modifying the configuration file, you need to recompile and source in the workspace:

```
cd ~/yahboom_ws
colcon build && source install/setup.bash
```

3.2 Start and Test the Function

1. Prepare the image file:

Place an image file to be tested in the following path:

```
~/yahboom_ws/src/largemodel/resources_file/visual_positioning
```

Then name the image `test_image.jpg`

2. Start the `largemodel` main program:

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py
```

3. Test:

- **Wake up:** Say to the microphone: "Hi,Yahboom."
- **Conversation:** After the speaker responds, you can say: `analyze the position of the dinosaur in the picture`
- **Observe the log:** In the terminal where the `launch` file is running, you should see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, calls the LLM, and prints the LLM's reply.
- **Listen to the answer:** After a while, you should be able to hear the answer from the speaker, and you will find an md document that records the coordinate position and positioning object information in the path `~/yahboom_ws/src/largemodel/resources_file/visual_positioning`.